

Старт проекта Telegram VPN Service (Marzban + Xray) с Claude Code

Обзор проекта и контекста

Проект представляет собой VPN-сервис, в котором управление доступом и биллингом для пользователей реализуются через Telegram-бота и Mini App, а транспортный уровень построен на Marzban (Xray-core) с протоколом VLESS + Reality. Marzban выступает панелью управления над Xray-core и изначально рассчитан на коммерческое использование и масштабирование до тысяч пользователей и множества нод.^{[1][2][^3]}

Бэкенд-часть бота планируется на Python с использованием aiogram 3.x, который использует router-ориентированную архитектуру и позволяет удобно модульно разносить обработчики обновлений по отдельным роутерам. Mini App строится на стеке React + Vite + Tailwind с использованием SDK для Telegram Mini Apps (@tma.js/sdk или аналогичный SDK), по аналогии с существующими boilerplate-проектами для Mini Apps.^{[4][5][6][7][8][9]}

Цель документа

Цель документа — дать пошаговый план старта разработки, разделив задачи на две части: что нужно делать вручную (как Product Owner/Admin), и что разумно делегировать Claude Code через хорошо сформулированный начальный промпт. Документ также предлагает целевую структуру репозитория, черновик файла CLAUDE.md с правилами разработки и структуру .env с учётом интеграции с Marzban и Telegram.

Базовые сведения о ключевых компонентах

Marzban — это панель управления прокси-серверами на базе Xray-core, написанная на Python (FastAPI) с фронтендом на React и использующая SQLAlchemy и реляционные базы данных для хранения данных. Панель позволяет централизованно управлять несколькими Xray-нодами, лимитами трафика, сроками действия подписок, а также поддерживает протоколы VLESS и Reality.^{[10][2][^3][1]}

Aiogram 3.x — современный асинхронный фреймворк для Telegram Bot API, который перешёл от старой модели Dispatcher к архитектуре Router, где обработчики регистрируются на отдельных роутерах и могут вкладываться друг в друга для модульной структуры проекта. Такая архитектура хорошо подходит для крупных ботов, где логика разделена по доменам (платежи, рефералы, тарифы, админка) и обеспечивает хорошую композицию и тестируемость кода.^{[6][8][^9]}

Telegram Mini Apps — это web-приложения, открывающиеся внутри Telegram, обычно разработанные на современных фронтенд-стеках типа React + Vite + Tailwind и интегрированные с Telegram через Mini App SDK. Существующие boilerplate-проекты демонстрируют типичную структуру: отдельная папка frontend/webapp, конфигурация Vite и Tailwind, интеграция SDK и скрипты npm для dev/build/deploy.^{[5][7][^4]}

Общая стратегия использования Claude Code

Claude Code удобнее всего использовать как «генератор репозитория и файлов» под ваши требования, а не как исполнителя команд на сервере. Он хорошо справляется с созданием структуры проекта, шаблонов файлов, моделей данных, обработчиков и фронтенд-компонентов, но не управляет реальной инфраструктурой, Docker-контейнерами или DNS.

Для получения максимальной пользы от Claude Code важно:

- Сначала явно описать контекст проекта, роли, стек и ограничения (Zero Logs, безопасность, платежи, протоколы).
- Сформулировать чёткую задачу: какие именно папки и файлы нужно создать, в каком формате, какие зависимости и версии использовать.
- Требовать, чтобы Claude всегда показывал структуру создаваемых файлов (дерево) и содержимое новых/изменённых файлов, а не только описания.

Что подготовить вручную до работы с Claude Code

До глубокой генерации кода через Claude Code полезно убедиться, что базовая инфраструктура и доступы готовы:

- **Marzban и Xray-core:**
 - Marzban развёрнут в Docker на VPS, панель доступна по HTTPS, доступны учётные данные администратора.^{[1][3]}
 - Настроены базовые inbound-правила Xray для VLESS + Reality, а также политика логирования с отключением access-логов (Zero Logs) в конфигурации Xray.^{[2][10]}
- **Telegram-бот и Mini App:**
 - Через @BotFather создан основной бот; получен BOT_TOKEN, настроено имя бота и описание.
 - Для Mini App подготовлен URL, который позже будет указывать на ваш Vite-frontend (на dev-этапе можно использовать https-линк с самоподписанным сертификатом, как рекомендуют шаблоны для Mini Apps).^{[4][5]}
- **База данных PostgreSQL:**
 - На VPS поднят PostgreSQL (либо планируется контейнер в docker-compose).
 - Созданы отдельный пользователь и база для бота, зафиксированы параметры подключения (host, port, db, user, password).
- **Платёжные провайдеры:**
 - Зарегистрированы аккаунты в Prodamus/Robokassa, получены тестовые и боевые ключи/логины и callback-секреты.
 - Определена минимальная модель биллинга: длительность тестового периода, тарифы, правила реферальной программы (2+1) и привязка к Telegram Stars (XTR).
- **Локальная среда разработки:**

- Установлен Python нужной версии (например, 3.11), Node.js (например, 18+), Docker и docker-compose.
- Настроен Git-репозиторий (локально или на GitHub/GitLab) для хранения кода.

Что делегировать Claude Code, а что делать самому

Задачи для Claude Code

Claude Code целесообразно поручить:

- Проектирование и создание структуры репозитория (bot/, webapp/, infra/, docs/ и т.д.).
- Генерацию минимальных рабочих шаблонов:
 - Бот на aiogram 3.x с разделением на routers и конфигом.
 - Минимальный React + Vite + Tailwind Mini App с интеграцией Telegram Mini App SDK.^[7]^[5]^[4]
 - Файл docker-compose.yml и Dockerfile-ы для бота, webapp и вспомогательных сервисов.
- Генерацию файла CLAUDE.md с правилами разработки, код-стайлом и оговоркой по Zero Logs.
- Генерацию структуры .env.example и использования этих переменных в коде (config.py, настройки подключения к БД, URL и ключи Marzban, токены Telegram, параметры платежей).
- Создание базовых моделей БД (SQLAlchemy) и миграций (Alembic), отражающих сущности: пользователь, подписка, тариф, реферальная связь, платежи.

Задачи, которые нужно делать вручную

Некоторые вещи должен делать именно владелец проекта:

- Хранить и наполнять реальные значения .env на сервере и локально; Claude должен оперировать только .env.example без секретов.
- Настраивать реальные домены, SSL-сертификаты, firewall, fail2ban и прочие элементы безопасности.
- Реально запускать docker-compose, проверять логи контейнеров, диагностировать сетевые проблемы.
- Заводить проекты и ключи в Prodamus/Robokassa, настраивать webhook-URL и сверять подписи.
- Принимать продуктовые решения: финальные тексты, UX Mini App, бизнес-правила реферальной программы, ценовые планы.

Рекомендуемая структура репозитория

Ниже приведён пример целевой структуры репозитория, которую можно попросить сгенерировать Claude Code:

```
.
├── bot/
```

```
|
| └─ app/
|   └─ __init__.py
|   └─ config.py
|   └─ main.py
|   └─ handlers/
|     └─ __init__.py
|     └─ start.py
|     └─ billing.py
|     └─ referrals.py
|     └─ admin.py
|   └─ keyboards/
|     └─ __init__.py
|     └─ common.py
|   └─ middlewares/
|     └─ __init__.py
|     └─ auth.py
|   └─ services/
|     └─ __init__.py
|     └─ marzban_client.py
|     └─ billing_prodamus.py
|     └─ billing_robokassa.py
|     └─ referrals.py
|   └─ db/
|     └─ __init__.py
|     └─ base.py
|     └─ models.py
|     └─ session.py
|   └─ pyproject.toml
|   └─ alembic.ini
|   └─ migrations/
| └─ webapp/
|   └─ src/
|     └─ main.tsx
|     └─ app/
|       └─ App.tsx
|       └─ routes.tsx
|     └─ components/
|       └─ layout/
|       └─ ui/
|       └─ vpn/
|     └─ pages/
|       └─ Home.tsx
|       └─ Pricing.tsx
|       └─ Referral.tsx
|       └─ Profile.tsx
|     └─ hooks/
|     └─ lib/
|       └─ telegram.ts
|     └─ styles/
|       └─ index.css
|   └─ index.html
|   └─ package.json
|   └─ tsconfig.json
|   └─ tailwind.config.cjs
|   └─ vite.config.ts
| └─ infra/
```

```

|   ├── docker-compose.yml
|   ├── nginx/
|   |   └── nginx.conf
|   ├── postgres/
|   |   └── init.sql
|   └── marzban/
|       └── README.md
├── docs/
|   └── architecture.md
├── CLAUDE.md
├── .env.example
└── README.md

```

Такая структура разделяет бот, фронтенд Mini App и инфраструктурные файлы, а также заранее выделяет места для бизнес-логики (services), интеграций (marzban_client, биллинги), конфигурации БД и документации.^{[5][2][^4]}

Структура файла .env

Ниже пример структуры .env.example, который имеет смысл попросить сгенерировать Claude Code и затем наполнить реальными значениями вручную:

```

# Общие настройки проекта
PROJECT_NAME=telegram-vpn-service
PROJECT_ENV=dev # dev/stage/prod
TIMEZONE=Europe/Berlin

# Telegram Bot
TELEGRAM_BOT_TOKEN=
TELEGRAM_BOT_USERNAME=
TELEGRAM_ADMIN_IDS=123456789,987654321 # список через запятую
TELEGRAM_LOG_CHAT_ID= # опционально, чат/канал для логов

# Telegram Mini App
TELEGRAM_MINIAPP_URL=https://vpn.example.com/app
TELEGRAM_MINIAPP_SHORT_NAME=your_mini_app

# База данных PostgreSQL
DB_HOST=postgres
DB_PORT=5432
DB_NAME=vpn_service
DB_USER=vpn_user
DB_PASSWORD=
DB_SSLMODE=prefer
DATABASE_URL=postgresql+psycopg://vpn_user:password@postgres:5432/vpn_service

# Marzban / Xray
MARZBAN_API_BASE_URL=https://marzban.example.com/api
MARZBAN_API_TOKEN=
MARZBAN_PANEL_URL=https://marzban.example.com/dashboard
MARZBAN_NODE_TAG=main

# Reality/VLESS (если нужно использовать на стороне бота для генерации ссылок)
XRAY_REALITY_PUBLIC_KEY=

```

```

XRAY_REALITY_SHORT_ID=
XRAY_REALITY_SERVER_NAME=
XRAY_PORT=443

# Платежи — Telegram Stars / Prodamus / Robokassa
TELEGRAM_STARS_TEST_MODE=true

PRODAMUS_SHOP_ID=
PRODAMUS_API_KEY=
PRODAMUS_SUCCESS_URL=https://vpn.example.com/payments/prodamus/success
PRODAMUS_FAIL_URL=https://vpn.example.com/payments/prodamus/fail
PRODAMUS_CALLBACK_SECRET=

ROBOKASSA_LOGIN=
ROBOKASSA_PASSWORD_1=
ROBOKASSA_PASSWORD_2=
ROBOKASSA_TEST_MODE=1
ROBOKASSA_RESULT_URL=https://vpn.example.com/payments/robokassa/callback
ROBOKASSA_SUCCESS_URL=https://vpn.example.com/payments/robokassa/success
ROBOKASSA_FAIL_URL=https://vpn.example.com/payments/robokassa/fail

# Логирование и мониторинг
LOG_LEVEL=INFO
SENTRY_DSN=

```

Важно, чтобы `.env.example` никогда не содержал реальные токены и пароли, а только ключи и примеры формата значений. Подключение к Marzban реализуется через его HTTP API, при этом сами настройки Xray (включая Zero Logs в Core Settings) остаются в панели и не хранятся в `.env` бота.^{[10][2]}

Черновик файла CLAUDE.md

Ниже пример содержания CLAUDE.md, который можно попросить Claude Code сгенерировать и затем при необходимости доработать:

```

# CLAUDE.md — правила для ИИ-разработчика

## Роль и контекст

- Ты — Senior Fullstack Developer & System Architect этого проекта.
- Проект: Telegram VPN Service на базе Marzban (Xray-core, VLESS + Reality), Telegram
- Моя роль: Product Owner & Admin, я отвечаю за инфраструктуру, домены, Docker, реал

## Общие правила работы

1. Всегда работай внутри текущего репозитория и текущей структуры пап

---

## References

1. [ReadmeX](https://readmex.com/en-US/Gozargah/Marzban/page-117766081-21d0-4460-be4
2. [Gozargah/Marzban: Unified GUI Censorship Resistant ...](https://github.com/Goza

```

3. [VPN Marzban: Профессиональная панель управления ...](<https://vc.ru/id3158218/2687>)
4. [supercoolx/telegram-mini-app-vite-react-javascript-tailwind - GitHub](<https://github.com/supercoolx/telegram-mini-app-vite-react-javascript-tailwind>)
5. [etherwave-labs/telegram-mini-app-react-boilerplate - GitHub](<https://github.com/etherwave-labs/telegram-mini-app-react-boilerplate>)
6. [Router - aiogram 3.16.0 documentation](<https://docs.aiogram.dev/en/v3.16.0/dispatcher/router.html>)
7. [@tma/sdk - JSR](<https://jsr.io/@tma/sdk>) - TypeScript library for seamless communication with Telegram Mini Apps
8. [Router - aiogram 3.21.0 documentation](<https://docs.aiogram.dev/en/latest/dispatcher/router.html>)
9. [Migration FAQ (2.x -> 3.0) - aiogram 3.21.0 documentation](<https://docs.aiogram.dev/en/latest/migration-faq.html>)
10. [Core Settings | Marzban - GitHub Pages](<https://gozargah.github.io/marzban/en/core-settings/>)
11. [Еще одна панель для управления проху на базе XRay-core - Пикабу](https://pikabu.ru/post_id/10000000000000000000/10000000000000000000)